

Управление 2D-ракетой методом MPC

Постановка задачи

Описание миссии

Двумерная модель ракеты должна выполнить миссию из трёх характерных точек: стартовать с поверхности, достичь заданной высоты h_{target} с малой вертикальной скоростью и затем мягко приземлиться в заданную точку x_{land} . **Профиль полёта между этими точками не задан** — контроллер MPC должен найти траекторию самостоятельно, исходя только из начального и конечного состояний, граничных условий промежуточной точки и динамики системы.

Управление осуществляется отклонением сопла и величиной тяги. Реализуется через скользящий горизонт с переключаемой целевой точкой (подход с двумя фазами: Ascent и Descent).

1. Динамическая модель

1.1. Переменные состояния и управления

Вектор состояния:

$$\mathbf{x} = [x \ y \ \dot{x} \ \dot{y} \ \vartheta \ \dot{\vartheta} \ \delta]^\top \in \mathbb{R}^7 \quad (1)$$

Вектор управления:

$$\mathbf{u} = [\delta_{\text{cmd}} \ F]^\top \in \mathbb{R}^2 \quad (2)$$

Обозначения:

x, y	координаты центра масс	м
$\dot{x}, \dot{y}$	компоненты скорости	м/с
ϑ	угол тангажа от вертикали (положительный вправо)	рад
$\dot{\vartheta}$	угловая скорость	рад/с
δ	текущий угол отклонения сопла	рад
δ_{cmd}	командный угол сопла	рад
F	величина тяги	Н

1.2. Уравнения движения

Поступательное движение (линеаризация по малому δ):

$$m\ddot{x} = F \sin \vartheta + F\delta \cos \vartheta + X_b \sin \vartheta + Y_b \cos \vartheta \quad (3)$$

$$m\ddot{y} = F \cos \vartheta - mg - F\delta \sin \vartheta + X_b \cos \vartheta - Y_b \sin \vartheta \quad (4)$$

Вращательное движение:

$$\ddot{\vartheta} = g_2 \delta + f_2, \quad g_2 = \frac{F l_{cp}}{J}, \quad f_2 = \frac{C_{m\alpha} \alpha q_\infty S_m l}{J} \quad (5)$$

Динамика привода сопла (апериодическое звено первого порядка):

$$\tau_\delta \dot{\delta} = -\delta + \delta_{cmd} \quad (6)$$

1.3. Дискретизация

С шагом Δt нелинейные уравнения движения интегрируются численно (например, методом RK4):

$$\mathbf{x}_{k+1} = \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k) \quad (7)$$

2. Граничные условия миссии

2.1. Начальное состояние

$$\mathbf{x}_0 = [x_{start} \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]^\top \quad (8)$$

2.2. Промежуточная точка

Задана **только высота**:

$$\mathbf{x}_{mid} : \quad y = h_{target}, \quad \text{остальные компоненты свободны} \quad (9)$$

2.3. Финальная точка

$$\mathbf{x}_f = [x_{land} \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]^\top \quad (10)$$

3. Машина состояний миссии

Две фазы

Ascent (взлёт): цель — промежуточная точка $\mathbf{x}_{mid}$.

Descent (посадка): цель — финальная точка $\mathbf{x}_f$.

Переключение Ascent $\rightarrow$ Descent срабатывает при выполнении условия:

$$|y - h_{target}| < \epsilon_h \quad \text{и} \quad |\dot{y}| < \epsilon_v \quad (11)$$

Терминация миссии при выполнении условия касания:

$$y < \epsilon_{land} \quad \text{и} \quad \sqrt{\dot{x}^2 + \dot{y}^2} < \epsilon_{vel} \quad (12)$$

4. Целевая функция МРС

4.1. Общий вид

На каждом такте t решается задача оптимизации на горизонте N шагов:

$$J = \underbrace{\sum_{k=0}^{N-1} \left[R_\delta \delta_{\text{cmd},k}^2 + R_F (F_k - F_{\text{hover}})^2 \right]}_{\text{stage cost: расход управления}} + \underbrace{(x_N - x_{\text{target}})^\top P (x_N - x_{\text{target}})}_{\text{терминальный штраф}} \quad (13)$$

где $F_{\text{hover}} = mg$ — тяга, необходимая для зависания при $\vartheta = 0$.

Целевая функция состоит из двух принципиально различных частей: *stage cost*, штрафующий расход управления на каждом шаге, и *терминального штрафа*, тянущего конец горизонта к целевой точке.

4.2. Stage cost: что штрафуются по пути

Сумма по всем шагам горизонта от $k = 0$ до $k = N-1$ штрафует **только управление** — ничего из состояния. Это сознательный выбор, и он принципиален для постановки задачи.

Слагаемое $R_\delta \delta_{\text{cmd},k}^2$

Квадратичный штраф на командный угол отклонения сопла. Назначение:

- **Гладкое управление.** Без штрафа оптимизатор мог бы выдавать рывки сопла: для него одинаково “дешево” постоянное $\delta_{\text{cmd}} = 0,5$ и переменное $\delta_{\text{cmd}} = \pm 1$ через шаг. С квадратичным штрафом второй вариант стоит вчетверо дороже — оптимизатор выберет первый.
- **Экономия ресурса актуатора.** Привод сопла изнашивается, большие отклонения требуют больше энергии. Штраф напрямую штрафует эту нагрузку.
- **Квадратичность важна.** Линеиный штраф $R_\delta |\delta_{\text{cmd}}|$ давал бы *разреженное* управление — предпочтение редким большим отклонениям. Квадратичный, наоборот, предпочитает много малых отклонений нескольким большим, что естественнее для угла сопла.

Слагаемое $R_F (F_k - F_{\text{hover}})^2$

Штрафуется **не сама тяга**, а её отклонение от уровня зависания $F_{\text{hover}} = mg$.

Почему так? Штраф вида $R_F F^2$ заставлял бы оптимизатор стремиться к $F = 0$ — физически невозможному (двигатель должен работать как минимум на $F_{\text{min}} > 0$) и нежелательному (ракета упадёт) режиму. Штрафуя отклонение от F_{hover} , формулировка фактически означает: “работай в режиме, близком к зависанию, и отклоняйся только когда нужно”.

Получаемое поведение. На взлёте требуется $F > F_{\text{hover}}$ — оптимизатор платит за это, но платит. В апогее и на спуске нужна меньшая тяга — снова отклонение, снова цена. В среднем тяга колеблется около F_{hover} .

Альтернативные формулировки:

- Для минимизации расхода топлива: $R_F \cdot F$ (линейный штраф), даёт bang-bang режим.
- Для минимизации импульса: $R_F \cdot F^2$ напрямую.

Что в stage cost намеренно отсутствует

В stage cost нет штрафов на состояние. Никаких членов вида $w_y(y_k - y_{\text{ref}})^2$. Это принципиально: в классическом tracking-MPC stage cost содержит штраф за отклонение от заданной траектории $(x_k - x_{\text{ref},k})^\top Q(x_k - x_{\text{ref},k})$. В данной постановке заданной траектории *нет* — добавление такого члена потребовало бы её придумать, что противоречит идее “MPC сам находит траекторию”.

4.3. Терминальный штраф

Член $(x_N - x_{\text{target}})^\top P(x_N - x_{\text{target}})$ штрафует отклонение **только в последней точке горизонта** от целевой точки. Для диагональной $P = \text{diag}(p_1, \dots, p_7)$:

$$(x_N - x_{\text{target}})^\top P(x_N - x_{\text{target}}) = \sum_{i=1}^7 p_i (x_{N,i} - x_{\text{target},i})^2 \quad (14)$$

Почему терминальный, а не интегральный штраф

Альтернатива — штрафовать отклонение на каждом шаге: $\sum_k (x_k - x_{\text{target}})^\top Q(x_k - x_{\text{target}})$. Это заставило бы ракету двигаться к цели каждый шаг — по сути, по прямой. Но прямая от старта к апогею не является оптимальной траекторией: реальная оптимальная траектория — баллистическая дуга, на которой ракета сначала улетает “не туда”, чтобы потом удачно приземлиться.

Терминальный штраф снимает это ограничение: оптимизатору всё равно, *как* аппарат попадёт в цель, важно лишь чтобы попал к концу горизонта. Это даёт максимальную свободу для нахождения оптимальной траектории.

Главный трюк: нули на свободных компонентах

В фазе Ascent промежуточная цель задаёт только высоту:

$$x_{\text{target}} = x_{\text{mid}}, \quad P_{\text{Ascent}} = \text{diag}(0, p_y, 0, p_{\dot{y}}, p_{\vartheta}, p_{\dot{\vartheta}}, 0) \quad (15)$$

Нули на компонентах x , $\dot{x}$ и δ означают: “мне всё равно, какая горизонтальная позиция в апогее, какая горизонтальная скорость и какое положение сопла”. Оптимизатор сам выберет эти значения так, чтобы было удобно для дальнейшей посадки.

Это мощный приём. Установка $p_x \neq 0$ навязала бы апогей в конкретной точке — “творческий” выбор, а не оптимум задачи. Оставляя нули, оптимизатору даётся максимальная свобода.

В фазе Descent всё фиксировано — точка посадки задана полностью:

$$\mathbf{x}_{\text{target}} = \mathbf{x}_f, \quad P_{\text{Descent}} = \text{diag}(p_x, p_y, p_{\dot{x}}, p_{\dot{y}}, p_{\vartheta}, p_{\dot{\vartheta}}, 0) \quad (16)$$

Только $p_{\delta} = 0$ — неважно положение сопла в момент касания.

4.4. Балансировка весов

Соотношение R и P определяет всё поведение системы:

- **Большой P , малый R** — “агрессивный” режим: тяга гуляет от $F_{\min}$ до $F_{\max}$, сопло дёргается, цель достигается быстро.
- **Малый P , большой R** — “ленивый” режим: управление бережётся, но цель достигается плохо или вообще не достигается за горизонт.
- **Сбалансированный** — цель достигается, управление разумное. Начинают с условных $R = 1$, $P = 100$ и подбирают.

Замечание о размерности весов

Слагаемые в стоимости должны быть сопоставимы по порядку величины. Если δ_{cmd} измеряется в радианах (значения $\sim 0,1$), а y — в метрах (значения ~ 1000), то $\delta_{cmd}^2 \sim 10^{-2}$, а $y^2 \sim 10^6$. Без нормировки второй член полностью “съест” первый.

Стандартный приём — нормировать на характерные масштабы:

$$P_{ii} = \frac{\bar{p}_i}{(x_i^{\text{scale}})^2}, \quad R = \frac{\bar{R}}{(u^{\text{scale}})^2}$$

Тогда $\bar{p}_i, \bar{R}$ — безразмерные веса порядка единицы, выражающие *относительную важность* компонент. Характерные масштабы задаются физикой задачи (h_{target} для высоты, $\delta_{\max}$ для угла и т. д.).

4.5. Переключение целевой функции между фазами

В момент переключения Ascent $\rightarrow$ Descent меняется как $\mathbf{x}_{\text{target}}$, так и матрица P . Это разрывное изменение функции стоимости.

Почему это работает на практике:

- Переключение происходит в “правильный” момент — когда ракета близка к промежуточной точке и динамика спокойная (вертикальная скорость ≈ 0).
- *Warm start* даёт оптимизатору хорошее начальное приближение от старого оптимума.
- После переключения оптимизатор за один-два такта сходится к новой траектории.

Альтернативы для гладкого перехода

Если требуется математически чистый переход без разрыва, есть варианты: **Soft transition** с весовой функцией $\alpha(t) \in [0, 1]$:

$$P(t) = (1 - \alpha(t)) P_{\text{Ascent}} + \alpha(t) P_{\text{Descent}}$$

где α плавно растёт от 0 до 1 в окрестности условия переключения.

Многоэтапная формулировка — включить обе точки в одну задачу:

$$J = \sum_k [\text{stage}] + (x_{k^*} - x_{\text{mid}})^\top P_1 (x_{k^*} - x_{\text{mid}}) + (x_N - x_f)^\top P_2 (x_N - x_f)$$

для некоторого промежуточного k^* . Переключения нет, но горизонт должен покрывать всю миссию.

В данной постановке проблема разрыва несущественна благодаря warm start и спокойному моменту переключения — усложнения не требуется.

5. Ограничения

Динамические:

$$x_{k+1} = f_d(x_k, u_k), \quad k = 0, \dots, N-1 \quad (17)$$

$$x_0 = x(t) \quad (18)$$

На управление:

$$|\delta_{\text{cmd},k}| \leq \delta_{\text{cmd,max}}, \quad F_{\min} \leq F_k \leq F_{\max} \quad (19)$$

На состояние:

$$|\vartheta_k| \leq \vartheta_{\max}, \quad |\delta_k| \leq \delta_{\max}, \quad y_k \geq 0 \quad (20)$$

6. Алгоритм МРС

Принцип скользящего горизонта

1. Измерить (или оценить) текущее состояние $x(t)$.
2. Определить текущую фазу и установить соответствующие x_{target} и P .
3. Решить задачу оптимизации, получить $u_0^*, \dots, u_{N-1}^*$.
4. Применить только u_0^* .
5. Обновить состояние, проверить условия переключения фазы и терминации.
6. Сдвинуть управляющую последовательность (warm start): $u_{\text{warm}} = (u_1^*, \dots, u_{N-1}^*, u_{N-1}^*)$.
7. Вернуться к шагу 1.

7. Параметры задачи

Физические параметры (константы):

$$m, J, l_{cp}, g, \tau_\delta, F_{\min}, F_{\max}, \delta_{\max}, \delta_{\text{cmd},\max}, \vartheta_{\max}$$

Параметры миссии:

$$x_{\text{start}}, x_{\text{land}}, h_{\text{target}}$$

Параметры MPC:

$$N, \Delta t, R_\delta, R_F, p_y, p_{\dot{y}}, p_\vartheta, p_{\dot{\vartheta}}, p_x, p_{\dot{x}}, \epsilon_h, \epsilon_v$$

8. Замечания о реализации

С учётом нелинейностей в $\sin \vartheta, \cos \vartheta$ задача является **нелинейной программой** (NLP). Для горизонта $N = 30$ это $2N$ переменных оптимизации с $7N$ динамическими ограничениями-равенствами и порядка $10N$ ограничений-неравенств — задача разумного размера, решаемая за десятки миллисекунд солверами IPOPT, SNOPT, ACADOS или CasADi.

Упрощённая версия (стартовый прототип): зафиксировать $\vartheta = 0$, отключить аэродинамику ($X_b = Y_b = f_2 = 0$), пренебречь динамикой привода ($\delta = \delta_{\text{cmd}}$). Получившаяся задача является **выпуклой квадратичной программой** (QP), решаемой за миллисекунды на каждом такте.

Ключевые свойства постановки:

- Нет заранее заданного профиля — траектория возникает как результат оптимизации.
- Промежуточная точка задана только по высоте — оптимизатор сам выбирает горизонтальное положение апогея.
- Дросселирование тяги $F \in [F_{\min}, F_{\max}]$ позволяет торможение при посадке.
- Машина состояний из двух фаз обеспечивает естественное разделение взлёта и посадки.
- Stage cost содержит только штрафы на управление — никаких “коридоров” в фазовом пространстве не навязано.
- Терминальный штраф с нулями на свободных компонентах в Ascent — ключевой приём, дающий оптимизатору максимальную свободу.